APL 1

Student Record Management System

Classes and Objects, Searching, Sorting and Grouping

1. Scenario

The examination cell of your college keeps a register of students — roll number, name, date of birth, and the PIN code of the home address. It needs to add students, list them, look one up by roll number, print the list in roll-number order, and group the students by PIN code.

Your job is to build that register in C++ using **two classes**.

2. The Two Classes

2.1 `class Student` — one record

Four **private** data members:

Data member	Type	Value
`rollNo`	`int`	Unique identifier
`name`	`string`	Single word, read with `cin >> name`
`dob`	`string`	Date of birth as `DD/MM/YYYY`
`pinCode`	`int`	Exactly six digits, $100000 \le$ `pinCode` ≤ 999999

Because the data members are private, nothing outside the class may touch them directly. `Student` therefore provides a constructor, a getter and a setter for each member, a `hasValidPinCode()` check, and two printing member functions — `print()` for a full table line and `printBrief()` for a line inside a PIN block.

2.2 `class StudentRecordSystem` — the register

This class **owns the data**. Its private section holds `Student records[100]` together with an `int count` saying how many slots are in use. That array is the only container of records in the whole program, and it is private, so the only way to reach a record is through a member function of this class.

Its public section has **one member function per menu option**, plus a `run()` function holding the menu loop.

Two small notes

- Names are **single words**, so `cin >> name` is enough — you do not need `getline`. If someone types `Ravi Kumar`, storing `Ravi` is acceptable here.
- An `int` cannot keep a leading zero, so `012345` would print as `12345`. This assignment avoids that by requiring the first digit to be non-zero, which is true of all Indian PIN codes.

3. What the Program Must Do

The program is a **menu loop**: print the menu, read one choice, do the operation, print a blank line, repeat. Only choice 7 exits.

#	What it does	What it prints
1	**Insert.** Read roll number, name, DOB and PIN code in that order. Check the rules in §4. Store the record at the **end** of the array.	`Student inserted successfully.` On failure: the matching error from §4, and nothing is stored.
2	**Display all**, in the **current stored order** — insertion order, unless option 4 has already reordered the array.	The table from §5, then `Total records: <n>` Empty: `No records to display.`
3	**Search by roll number.** Report the **0-based index** at which the record was found.	`Record found at position <i>.` then that one table row Absent: `Record not found for roll number <r>.` Empty: `No records to search.`
4	**Sort by roll number**, ascending, **inside the same array**. The new order is **permanent**. Use any algorithm — bubble, selection or insertion sort — and name it in a comment.	`Records sorted by roll number.` then the table Empty: the message, then `No records to display.`
5	**Segregate by PIN code.** This is a **report** — it must **not** rearrange the array. Print the groups in **ascending PIN order**; inside a group, print in stored order.	One block per PIN code (§5), with a blank line between blocks Empty: `No records to segregate.`
6	**Count students per PIN code**, in ascending PIN order, each PIN code listed once.	`<pin> -> <count> students` per line, then `Distinct PIN codes: <k>` Empty: `No records to count.`
7	**Exit.**	`Exiting program. Goodbye!` then `return 0;`

Any choice outside 1–7 prints `Invalid choice. Please try again.` and shows the menu again.

4. Validation Rules

Only three checks, all inside option 1. Print each message **exactly** as written — that is what makes the test cases checkable.

Check	Rule	Message on violation
Capacity	At most 100 records	`Error: Database is full. Cannot insert.`
Roll number	Must not already be in the array	`Error: Roll number already exists.`
PIN code	Integer from 100000 to 999999	`Error: PIN code must be exactly 6 digits.`

How to apply them

Check capacity first, then read the roll number and check for a duplicate **before** asking for name, DOB and PIN code. On any violation, print that one message, store nothing, and go back to the menu — do not ask again.
The date of birth is **not** validated: store whatever the user types. Checking it is a bonus (§??).

5. Output Format

The menu, the record table, and one PIN block

```
==========================================
       STUDENT RECORD MANAGEMENT
==========================================
1. Insert Student
2. Display All Students
3. Search by Roll Number
4. Sort by Roll Number
5. Segregate by PIN Code
6. Count Students by PIN Code
7. Exit
==========================================
Enter your choice:


Roll No    Name            DOB             PIN Code
--------------------------------------------------
101        Rahul           12/05/2004      500081


PIN CODE : 500072
-----------------
103        Arjun           21/11/2004
110        Sneha           17/09/2005
```

Column alignment is **not graded** — separating fields with `"\t"` is fine, though `setw` from `<iomanip>` looks neater. What **is** graded: the order of the lines, which lines appear, and the exact wording of the status and error messages.
In option 6 always write the plural `students`, even when the count is 1. Do not special-case it.

6. Starter Code

Copy this into your `.cpp` file. Both class definitions, `run()` and `main()` are already written. **Your work is to write the bodies of the member functions declared below** — define them outside the class using the `ClassName::functionName` form.

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

// ==================== class Student : one record ====================
class Student {
private:
    int rollNo; // unique
    string name; // single word
    string dob; // "DD/MM/YYYY"
    int pinCode; // 100000 .. 999999

public:
    Student() : rollNo(0), name(""), dob(""), pinCode(0) {}
    Student(int r, const string &nm, const string &d, int p)
        : rollNo(r), name(nm), dob(d), pinCode(p) {}

    // getters -- const, because they promise not to modify the object
    int getRollNo() const { return rollNo; }
    string getName() const { return name; }
    string getDob() const { return dob; }
    int getPinCode() const { return pinCode; }

    // setters
    void setRollNo(int r) { rollNo = r; }
    void setName(const string &nm) { name = nm; }
    void setDob(const string &d) { dob = d; }
    void setPinCode(int p) { pinCode = p; }

    bool hasValidPinCode() const; // YOU WRITE: six digits?
    void print() const; // YOU WRITE: one full table line
    void printBrief() const; // YOU WRITE: roll, name, DOB only
};

// ========== class StudentRecordSystem : owns all the records ==========
class StudentRecordSystem {
private:
    static const int MAX = 100;

    Student records[MAX]; // THE one container -- private, so it is protected
    int count; // how many slots are in use

    // private helpers -- callable only from inside this class
    int findIndex(int roll) const; // YOU WRITE: index, or -1
    int collectSortedPins(int pins[]) const; // YOU WRITE: distinct PINs, ascending
    static void printMenu(); // YOU WRITE
    static void printHeader(); // YOU WRITE

public:
    StudentRecordSystem() : count(0) {} // starts empty
```

```
    // one public member function per menu option -- YOU WRITE ALL SIX
    void insertStudent();
    void displayStudents() const;
    void searchStudent() const;
    void sortByRollNo();
    void segregateByPincode() const;
    void countByPincode() const;

    void run(); // the menu loop (given below)
};

// ---- the menu loop : already written for you ----
void StudentRecordSystem::run() {
    int choice;
    while (true) {
        printMenu();
        if (!(cin >> choice)) return; // stop cleanly if the input ends
        switch (choice) {
            case 1: insertStudent(); break;
            case 2: displayStudents(); break;
            case 3: searchStudent(); break;
            case 4: sortByRollNo(); break;
            case 5: segregateByPincode(); break;
            case 6: countByPincode(); break;
            case 7: cout << "Exiting program. Goodbye!\n"; return;
            default: cout << "Invalid choice. Please try again.\n";
        }
        cout << "\n";
    }
}

int main() {
    StudentRecordSystem app;
    app.run();
    return 0;
}
```

7. Test Cases

TC-1 to TC-5 are **one continuous session** — each starts from the state the previous one left behind. TC-6 is a fresh run. The menu banner is left out below to save space.

Base data, inserted in this order in TC-1:

Roll No	Name	DOB	PIN Code
101	Rahul	12/05/2004	500081
105	Priya	08/01/2005	500081
103	Arjun	21/11/2004	500072
110	Sneha	17/09/2005	500072
108	Kiran	05/03/2004	500090

7.1 TC-1 — Insert five records, then display

TC-1

```
Enter your choice: 1
Enter Roll Number: 101
Enter Name: Rahul
Enter Date of Birth (DD/MM/YYYY): 12/05/2004
Enter PIN Code: 500081
Student inserted successfully.

   ... repeat for 105 Priya, 103 Arjun, 110 Sneha, 108 Kiran ...

Enter your choice: 2

Roll No   Name            DOB             PIN Code
--------------------------------------------------
101       Rahul           12/05/2004      500081
105       Priya           08/01/2005      500081
103       Arjun           21/11/2004      500072
110       Sneha           17/09/2005      500072
108       Kiran           05/03/2004      500090
Total records: 5
```

This tests: records come out in *insertion* order, not sorted order.

7.2 TC-2 — The two validation checks

TC-2 — two rejected inserts

```
Enter your choice: 1
Enter Roll Number: 103
Error: Roll number already exists.

Enter your choice: 1
Enter Roll Number: 115
Enter Name: Divya
Enter Date of Birth (DD/MM/YYYY): 22/08/2005
Enter PIN Code: 50008
Error: PIN code must be exactly 6 digits.

Enter your choice: 2
   ... table unchanged, Total records: 5 ...
```

This tests: the duplicate is caught *before* the name, DOB and PIN code are asked for — the program must not prompt for them at all. Neither attempt stores anything, so the count stays at 5.

7.3 TC-3 — Search, found and not found

TC-3

```
Enter your choice: 3
Enter Roll Number to search: 110
Record found at position 3.

Roll No    Name            DOB             PIN Code
--------------------------------------------------
110        Sneha           17/09/2005      500072

Enter your choice: 3
Enter Roll Number to search: 999
Record not found for roll number 999.
```

This tests: the position is **0-based** — Sneha was the 4th record inserted, so index 3.

7.4 TC-4 — Sort, and the new order stays

TC-4

```
Enter your choice: 4
Records sorted by roll number.

Roll No   Name            DOB             PIN Code
-------------------------------------------------
101       Rahul           12/05/2004      500081
103       Arjun           21/11/2004      500072
105       Priya           08/01/2005      500081
108       Kiran           05/03/2004      500090
110       Sneha           17/09/2005      500072
Total records: 5

Enter your choice: 3
Enter Roll Number to search: 110
Record found at position 4.
   ... Sneha's row ...
```

This tests: you sorted the array itself. Roll 110 now reports position 4, not 3. A program that prints a sorted copy but leaves the array alone fails here.

7.5 TC-5 — Segregate and count

TC-5

```
Enter your choice: 5

PIN CODE : 500072
-----------------
103       Arjun           21/11/2004
110       Sneha           17/09/2005

PIN CODE : 500081
-----------------
101       Rahul           12/05/2004
105       Priya           08/01/2005

PIN CODE : 500090
-----------------
108       Kiran           05/03/2004

Enter your choice: 6
500072 -> 2 students
500081 -> 2 students
500090 -> 1 students
Distinct PIN codes: 3
```

This tests: the PIN groups come out in **ascending order** — 500072, then 500081, then 500090 — *not* in the order they were first seen. Each PIN code appears **once**, so you must skip PIN codes you have already handled. Note `1 students`: always plural, as specified.

Why the rows inside each group look sorted: because TC-4 already sorted the array. Inside a group you just print in stored order — **no second sort is needed**.

7.6 TC-6 — Every option on an empty database

TC-6 — fresh run, nothing inserted

```
Enter your choice: 2
No records to display.

Enter your choice: 3
Enter Roll Number to search: 101
No records to search.

Enter your choice: 4
Records sorted by roll number.
No records to display.

Enter your choice: 5
No records to segregate.

Enter your choice: 6
No records to count.

Enter your choice: 0
Invalid choice. Please try again.

Enter your choice: 7
Exiting program. Goodbye!
```

This tests: no crash, no garbage, no empty table header. This is where programs fail most often. Put an `if (count == 0)` guard at the top of every operation, before you touch `records[0]`.

7.7 The capacity check

You do not need to type 100 records. Temporarily change `const int MAX = 100;` to `3`, insert 3 records, try a 4th, and confirm you get `Error: Database is full. Cannot insert.` Then set it back to 100.

Tip. Put the TC-1 to TC-5 keystrokes into `input.txt` and run `a.exe < input.txt` (Windows) or `./a.out < input.txt` (Linux/macOS). You can then replay the whole session after every fix instead of retyping it.

8. Rules

Breaking these loses marks

- **Both classes are used as given.** Do not turn either one back into a `struct` and do not add a second class that stores records.
- **All data members stay `private`**, in both classes. Reach a record's fields only through its getters and setters. Making anything `public` to save typing loses the encapsulation marks.
- Records are stored in **one array of objects**, `records[MAX]`, nothing else. A small local `int` array inside a member function is fine.
- **Keep the `const`** on the four read-only member functions.
- At most **100** records, enforced by your own check.
- Roll numbers are **unique**.
- **One member function per operation.** `main()` stays exactly as given — create the object, call `run()`, return.
- The program must never crash on an empty database or a missing roll number.
- Every member function has a one-line comment saying what it does.

9. Before You Submit

Tick every line

- ☐ Compiles with `g++ -std=c++11 -Wall` and gives no warnings.
- ☐ TC-1 to TC-6 all reproduce, with the exact error messages.
- ☐ Every data member in both classes is still `private`.
- ☐ The four read-only member functions still say `const` — on the declaration *and* on the definition.
- ☐ `main()` is still three lines.
- ☐ The menu comes back after every operation; only choice 7 exits.
- ☐ Every option is safe when `count == 0` (TC-6).
- ☐ Sorting changes `records[]` itself and the order stays (TC-4).
- ☐ Segregation is in ascending PIN order and prints each PIN once (TC-5).
- ☐ Segregation did *not* rearrange the array — run option 2 afterwards and confirm the order is unchanged.
- ☐ Every member function has a comment.

10. Submission

1. `<RollNo>_StudentRecords.cpp`, with a header comment giving your name, roll number and the date..

11. Common Mistakes

Seen every semester

1. **Forgetting `StudentRecordSystem::` on a definition written outside the class.** You then get a plain global function, and the linker complains that the member function was never defined.
2. **Putting `const` on the declaration but not the definition** (or the other way round). Those are two *different* functions to the compiler, so again the member function is never defined.
3. Making the data members `public` to avoid writing getters. It compiles, but it throws away the whole point of the exercise.
4. Trying to modify `records[]` or `count` inside a `const` member function. The compiler is right and you are wrong — if option 5 needs to change the array, your design is wrong, not the `const`.
5. Printing the table header when there are zero records.
6. Sorting into a temporary copy, so the next Display shows the old order.
7. Printing the same PIN code group two or three times, because you forgot to skip PIN codes already handled.
8. "Segregating" by sorting the array by PIN code — that quietly destroys the roll-number order the user asked for in option 4.
9. Forgetting to increase `count` after an insert, or increasing it even when the insert was rejected.
10. Reading the PIN code before checking the duplicate roll number, so TC-2 prompts for fields it should not.